Continued Interactive GSLTs and the Cost Endofunctor

A construction one level up from cost-accounted Rholang

(revised: wrapping by construction)

L. G. Meredith*

F1R3FLY.io

May 2026

Abstract

We lift the cost-accounting construction of the rho calculus from a single calculus to an endofunctor on a category of interactive graph-structured lambda theories (GSLTs). The earlier work folded phlogiston accounting into the grammar of Rholang by signing terms and replacing the communication rule by a family of rules that rewrite *signed* terms while consuming *tokens*. Here we observe that this is a generic procedure governed by a single shape of dynamics, the *interaction cut*: an interaction constructor K brings two introductions $K_{\mathrm{p}}(I, C_{\mathrm{p}})$ and $K_{\mathrm{e}}(J, C_{\mathrm{e}})$ into contact along matching *interaction surfaces* I, J, and a contraction $\mathsf{compute}(I, J, C_{\mathrm{p}}, C_{\mathrm{e}})$ — Milner's pseudo-application in the process-calculus instances — combines their *program* and *environment* continuations $C_{\mathrm{p}}, C_{\mathrm{e}}$. Binding is one mode by which information migrates between environment and program under this contraction, not an intrinsic feature of the cut: CCS realises the same shape with no migration at all, and is in this sense the paradigmatic instance. An interactive GSLT whose dynamics admits this presentation, together with a computable section of its structural-congruence quotient and a contraction compatible with wrapping, we call a *continued* interactive GSLT. The point of the adjective is not that such a theory is already metered or wrapped — a continued interactive GSLT is an ordinary, un-metered calculus — but that it carries exactly the structure needed for the cost construction to *build* a metered version from it.

That construction is the endofunctor $\mathfrak{C}$. Given a continued interactive GSLT, $\mathfrak{C}$ produces a calculus in which the continuation slots are *wrapped terms by construction*: every continuation is a metered thunk in the grammar of the image. The decisive design move is that this wrapping lives in the *output*, so that "every redex sits inside a wrapper" is a sorting invariant of $\mathfrak{C}(G)$ preserved by reduction — not a property that must be re-established by traversing each contractum. The source theory's contraction need not be lifted; in the image it already maps wrapped terms to wrapped terms, and the only run-time interaction with tokens is *unwrapping*: forcing a thunk by spending a matching token. Metering is thereby *lazy* — work is charged when performed, not when exposed — and the temporal token stack is consumed exactly in step with the reductions, realising the modulus of the underlying cut elimination as the run length itself.

We show that $\mathfrak{C}$ is an endofunctor on the category **ciGSLT** of continued interactive GSLTs with bisimulation-preserving, quote-faithful morphisms. Throughout we keep **ciGSLT** formally distinct from the ambient category **iGSLT** of interactive GSLTs: a forgetful functor U : **ciGSLT** $\to$ **iGSLT** exhibits "continued" as a proper structural enrichment of "interactive," so that the endofunctor's domain pins down exactly which interactive theories admit sound,

*lgreg.meredith@f1r3fly.io

lazy cost accounting. Finally, iterated cost accounting flattens — nested signatures multiply and stacks-of-stacks concatenate — so $\mathfrak{C}$ is a (non-idempotent) monad, resolving through two adjunctions with opposite embeddings: a structural free–forgetful adjunction that detaches the apparatus, and, over a Turing-complete base, an internalisation adjunction that dissolves the apparatus into the base's own computation up to weak bisimulation. We close with refinements that make the metered calculus a usable resource discipline — object capabilities, an opt-in type discipline for token usage, and resource-sufficiency proofs — and with some Curry–Howard-like correspondences to physical theories, offered for further investigation: K as space and $::$ as time, with cut elimination coupling an information distribution to the nearness structure in the manner of Einstein's field equations; and token distributions as fields whose complex weights recover quantum-field-theoretic interference.

Contents

1 Introduction

In previous work we showed that the cost-accounting layer of Rholang — the phlogiston (phlo) system — need not be a privileged runtime extension. By treating signatures as a species of name, token stacks as first-class terms, and the for/send operators as ranging over *signed* terms, we folded cost accounting directly into the grammar and operational semantics of the rho calculus [1]. The single communication rule Comm became a family of rules that gate communication on the consumption of a matching token. Because the rho calculus is the formal core of Rholang, this revised calculus serves as the design specification for rearchitecting phlo accounting in Rholang 1.4. The rho calculus is a member of the family of mobile process calculi [5, 9], distinguished by its reflective treatment of names as quoted processes [1, 18].

This note takes the construction *one level up*, asking what the move requires of the underlying theory and to which theories it applies. Two observations drive the development. First, the dynamics must be presented as an *interaction cut*: two introductions meet along matching interaction surfaces, each carrying its own continuation — a program continuation and an environment continuation — which a contraction then combines. This is visible already in Milner's polyadic π-calculus tutorial [6], where the input $\mathrm{for}(y \leftarrow x)P$ is the pair $(x, \lambda y.P)$ and the output $x!(u).Q$ is the pair $(x, [u, Q])$: the subjects x are the surfaces that must match, the objects $\lambda y.P$ and $[u, Q]$ are the continuations, and the contraction is Milner's *pseudo-application*, returning $P\{u/y\} \mid Q$. Second — and this is the heart of the revision — a theory presented in this form carries enough structure for the cost endofunctor to build a metered version in which the continuation slots are *wrapped terms by construction*. We are careful to keep the two apart: the underlying theory (a *continued* interactive GSLT) is an ordinary un-metered calculus that merely *admits* the construction; the wrapping is a feature of the *image* $\mathfrak{C}(G)$, not of G. In that image every continuation is a metered thunk in the grammar, so the contractum is born guarded: no traversal re-wraps it, the contraction is not lifted, and the absence of leaks is a sorting invariant of $\mathfrak{C}(G)$ rather than a dynamic obligation. The run-time semantics of the metered calculus only ever *unwraps*, forcing a thunk by spending a token.

The construction spans calculi with and without binding, and we use several as foils. CCS [7] is the paradigmatic continued interactive GSLT: its interaction $a.P \mid \bar{a}.Q \to P \mid Q$ is an interaction cut with *no* binding and no information migration — the surfaces are complementary labels, and pseudo-application degenerates to releasing the two continuations. The interaction categories of Abramsky, Gay, and Nagarajan [8] generalise this further, with interaction as composition along a shared interface. The rho calculus and the λ-calculus sit at the binding end of the spectrum, where pseudo-application substitutes. In the rho calculus several distinct structures are fused into $\mid$ and $@\cdot$; pulling them apart in λ, where they are visibly separate, exposes the construction as a genuine endofunctor and identifies precisely what an interactive theory must supply. Binding, on this view, is one way information migrates between environment and program — not what makes an interaction a cut.

Section 2 isolates the interaction cut. Section 3 introduces wrapping by construction, derives no-leak as subject reduction, and contrasts lazy with eager metering. Section 4 identifies the spatial monoid K and the temporal monoid $::$, with the stack as a reified clock and lazily extracted modulus. Section 5 treats signature construction as a section of the structural-congruence quotient, with de Bruijn canonicalisation supplying it for λ. Section 6 assembles the endofunctor $\mathfrak{C}$ on **ciGSLT**. Section 7 records the graded adequacy statement. Section 8 works the two anchoring instances. Section 9 observes that iterated cost accounting flattens, so that $\mathfrak{C}$ is a monad, and exhibits its two resolutions: a structural free–forgetful adjunction and, over Turing-complete bases, a computational internalisation adjunction. Section 10 relates the construction to object capabilities, showing that whether proximity to a resource stack confers authority depends on the equational theory of K, and refining the stack to be *located* at an interaction surface. Section 11 uses OSLF on the cost-accounted theory as an opt-in type discipline for token usage, expressed through spatial and modal connectives. Section 12 lifts the linear-proof guarantee of resource sufficiency, examines its fate under data-dependent interaction, and shows that located purses make the guarantee local and compositional.

2 The interaction cut

2.1 Interactive GSLTs

A *graph-structured lambda theory* is a multi-sorted term theory $G = (\Sigma, E, R)$: a signature Σ of sorts and operators, a set E of equations imposing a structural congruence $\equiv$, and a set R of rewrite rules. The framing of operational semantics in this algebraic, graph-enriched style follows Stay and Meredith and Baez and Williams [3, 4]. It is *interactive* when it carries a distinguished interacting sort P with a labelled transition system, generated by R modulo $\equiv$, on which bisimulation is the intended equivalence. CCS [7] (P = processes, interaction by complementary actions), the rho calculus (P = processes, $\equiv$ the $\mid$-monoid congruence with α, R = Comm), the λ-calculus (P = terms, $\equiv = \alpha$, $R = \beta$), and the interaction categories of Abramsky, Gay, and Nagarajan [8] are all motivating examples — spanning, as we will see, calculi with and without binding.

Definition 2.1 (The category **iGSLT**)**. iGSLT** is the category whose objects are interactive GSLTs and whose morphisms are *theory maps* — sort- and operator-preserving translations that respect $\equiv$ and R — that are **bisimulation-preserving** on the interacting sort.

This is the base category of the present note. The cost construction does *not* act on all of **iGSLT**; it acts on the objects that carry enough additional structure to be metered, which we isolate as a subcategory in Section 6. Keeping the two apart is deliberate: "interactive" is the ambient setting, while "continued" (Definition 6.1) is the structural condition under which sound,

lazy cost accounting exists. The distinction is what the endofunctor's domain makes precise — an interactive GSLT is a place where processes interact; a continued interactive GSLT is one whose interaction is presented as a meterable interaction cut.

2.2 The interaction-cut presentation

The cost construction acts on dynamics presented in a particular shape. Write the generating rule of R as

$$\frac{(C'_{\mathrm{p}},\, C'_{\mathrm{e}}) \;=\; \mathsf{compute}(I,\, J,\, C_{\mathrm{p}},\, C_{\mathrm{e}})}{K\big(K_{\mathrm{p}}(I,\, C_{\mathrm{p}}),\; K_{\mathrm{e}}(J,\, C_{\mathrm{e}})\big) \;\longrightarrow\; K'\big(C'_{\mathrm{p}},\, C'_{\mathrm{e}}\big)} \qquad (\dagger)$$

distinguishing:

the interaction constructor K, which brings two operands into *contact*. In rho, $K = |$; in λ, $K = \mathrm{App}$; in CCS, $K = |$.

two introductions $K_{\mathrm{p}}(I, C_{\mathrm{p}})$ and $K_{\mathrm{e}}(J, C_{\mathrm{e}})$, each a constructor separating an *interaction surface* $(I,\, J)$ — the part that must match for the cut to fire — from a *continuation*: C_{p} the *program continuation* (the rest of the computation) and C_{e} the *environment continuation* (the rest of the environment). A surface may be carried *nominally*, as an explicit I or J that must match (as the subject x does in rho), or *structurally*, by the rigidity of K: when K is *free* (carries no equations), position itself is unforgeable and the context $K([\,], -)$ *is* the surface, so a nominally *degenerate* (absent) surface is not a missing match condition but one carried by the geometry of K instead. λ is exactly this case on the environment side — App is free, and its rigidity supplies the surface that no explicit J records. The two carriers are developed in Section 10. The factoring is Milner's [6]: in rho and π, the input $\mathrm{for}(y \leftarrow x)P$ is the pair $(x, \lambda y.P)$ and the output $x!(u).Q$ is the pair $(x, [u, Q])$, so $I = x = J$ are the matching subjects while $C_{\mathrm{p}} = \lambda y.P$ and $C_{\mathrm{e}} = [u, Q]$ are the continuations. In CCS the surfaces are complementary actions $a, \bar{a}$ and the continuations carry no data.

the contraction $\mathsf{compute}$, a partial operation that, on a surface match, combines the two continuations into residuals $C'_{\mathrm{p}}, C'_{\mathrm{e}}$ repackaged by K'. This is Milner's *pseudo-application* [6]: applied to $\lambda y.P$ and $[u, Q]$ it returns $P\{u/y\} \mid Q$. It factors into two parts — the substitution $P\{u/y\}$, by which the datum u *migrates* from the environment continuation into the program continuation through the binder y, and the release $\mid Q$ of the environment's residual.

We call ($\dagger$) an *interaction cut*: two introductions meet along matching surfaces and a contraction combines their continuations. Both sides are structured — the eliminand is not a bare term but an introduction carrying its own continuation, exactly as $x!(u).Q$ in rho is the pair $(x, [u, Q])$ rather than a bare datum. We avoid the word *symmetric* for this: the cut need not be commutative (in λ it is not), and whether it is commutative is a separate property of the equational theory of K, treated in Section 10. *Binding is one mode of information migration*, not a feature of the cut as such: it is the substitution half of pseudo-application, present in rho, π, and λ, absent in CCS, where pseudo-application degenerates to pure release. The λ-calculus is the further degenerate case in which the environment introduction K_{e} has a degenerate (absent) surface and the environment continuation carries no residual, so that the argument is its own continuation and pseudo-application is ordinary application (Section 8).

Remark 2.2 (K is the only constructor that needs overloading)**.** The cut isolates one contact site. Every cost rule interposes a token between the operands of K and drains it on contact. There is one K per cut, hence one constructor to make polymorphic. "Meter the cut" is the generic content of "place valuable friction at every interaction."

3 Wrapping by construction

This section describes what the cost endofunctor *builds*. Given a continued interactive GSLT G — an un-metered calculus presented as a interaction cut, with a section and a wrapping-compatible contraction (Section 6) — the endofunctor $\mathfrak{C}$ produces a metered calculus $\mathfrak{C}(G)$ whose grammar makes every continuation a metered thunk. The wrapping discipline below is therefore a property of the image $\mathfrak{C}(G)$; the source G supplies only the structure that makes the discipline installable.

3.1 Continuations are metered thunks in the image

In $\mathfrak{C}(G)$ the cost endofunctor adjoins a sort Sig of signatures (Section 5), a *wrapped-term* sort $\mathbb{T}$ with the wrapper $\{\cdot\}_s : P \times \mathsf{Sig} \to \mathbb{T}$, and a sort Stk of token stacks

$$S \;::=\; () \;\mid\; s :: S, \qquad s \in \mathsf{Sig}. \tag{1}$$

The pivotal discipline is grammatical, and it is imposed by $\mathfrak{C}$ on its image: **in $\mathfrak{C}(G)$ every continuation slot of K_{p} and K_{e} has sort $\mathbb{T}$.** Structural composition (parallel composition in rho) lives at the $\mathbb{T}$ level, between wrappers, so that each redex-bearing thread is individually wrapped. Thus a continuation in the metered calculus is not a bare process awaiting metering; it is a *metered thunk* — $\{P\}_s$ — whose activation is gated by a token keyed to s, and whose own continuations are again thunks. (In the source G these slots carried ordinary terms; re-sorting them to $\mathbb{T}$ is part of what $\mathfrak{C}$ does, Construction 6.6.) Reading $C_{\mathrm{p}} = \{P\}_{s_{\mathrm{p}}}$ and $C_{\mathrm{e}} = \{Q\}_{s_{\mathrm{e}}}$, the contraction of the image has the type

$$\mathsf{compute} \;:\; \mathit{Surf} \times \mathit{Surf} \times \mathbb{T} \times \mathbb{T} \;\longrightarrow\; \mathbb{T} \times \mathbb{T},$$

mapping wrapped continuations to wrapped continuations. It need not know that wrapping exists: substituting a wrapped term into a position yields a wrapped term, so $\mathsf{compute}$ preserves the sort $\mathbb{T}$ for free.

3.2 The gated rule: force by unwrapping

A wrapped redex is forced by consuming a matching token. The single-signature rule seals the whole redex under one s and funds it from a purse located at the surface $L = \mathsf{near}(I, J)$, with near reading the surfaces I, J through the seal (at the digest level, without forcing the continuations):

$$\frac{L = \mathsf{near}(I,J), \quad (\{C'_{\mathrm{p}}\}_{t_1}, \{C'_{\mathrm{e}}\}_{t_2}) = \mathsf{compute}(I, J, \{C_{\mathrm{p}}\}_{t_1}, \{C_{\mathrm{e}}\}_{t_2})}{K\big(\{K(K_{\mathrm{p}}(I, \{C_{\mathrm{p}}\}_{t_1}),\, K_{\mathrm{e}}(J, \{C_{\mathrm{e}}\}_{t_2}))\}_s,\; S(L, s :: p)\big) \;\longrightarrow\; K\big(K'(\{C'_{\mathrm{p}}\}_{t_1}, \{C'_{\mathrm{e}}\}_{t_2}),\; S(L,p)\big)} \tag{R1}$$

in which the outer polymorphic K combines the wrapped redex with the located purse, the head cell s is consumed (the act of *unwrapping* the redex), and the residual $K'(\{C'_{\mathrm{p}}\}_{t_1}, \{C'_{\mathrm{e}}\}_{t_2})$ is — by the typing of $\mathsf{compute}$ — already built from wrapped terms, the purse remaining located at L. There is no re-wrapping step and no lifted contraction. The remaining cases seal the two interaction *surfaces* rather than the whole redex, exposing them directly; they share the premise

$$(\{C'_{\mathrm{p}}\}_{t_1},\, \{C'_{\mathrm{e}}\}_{t_2}) = \mathsf{compute}\big(I,\, J,\, \{C_{\mathrm{p}}\}_{t_1},\, \{C_{\mathrm{e}}\}_{t_2}\big), \qquad L = \mathsf{near}(I, J),$$

in which the surface seals s_1, s_2 fund the cut while the continuation keys t_1, t_2 are carried unchanged through compute (no re-wrapping):

$$K\big(K(K_{\mathrm{p}}(\{I\}_{s_1},\,\{C_{\mathrm{p}}\}_{t_1}),\;K_{\mathrm{e}}(\{J\}_{s_2},\,\{C_{\mathrm{e}}\}_{t_2})),\;S(L, s_1 * s_2 :: p)\big)$$
$$\longrightarrow\; K\big(K'(\{C'_{\mathrm{p}}\}_{t_1},\,\{C'_{\mathrm{e}}\}_{t_2}),\;S(L,p)\big), \tag{R2}$$

$$K\big(K(K_{\mathrm{p}}(\{I\}_{s_1},\,\{C_{\mathrm{p}}\}_{t_1}),\;K_{\mathrm{e}}(\{J\}_{s_2},\,\{C_{\mathrm{e}}\}_{t_2})),\;K(S(L, s_1 :: p_1),\,S(L, s_2 :: p_2))\big)$$
$$\longrightarrow\; K\big(K'(\{C'_{\mathrm{p}}\}_{t_1},\,\{C'_{\mathrm{e}}\}_{t_2}),\;K(S(L,p_1),\,S(L,p_2))\big). \tag{R3}$$

The polymorphism of $K_{\mathrm{p}}, K_{\mathrm{e}}$ over their surface slots (Construction 6.6) is what lets R1 and R2/R3 coexist as one rule family: R1 fills the surface slot with a bare surface and seals the whole redex, while R2/R3 fill it with a *sealed* surface $\{I\}_{s_1}$ and fund per surface. In both R2 and R3 the two surfaces are sealed under s_1, s_2 and consumed by the matching purse cell, the continuations sealed under t_1, t_2 are preserved by compute, and the purse remains located at L across the step — the temporal counterpart of well-wrapping, preserved alongside it.

3.3 No leak, as a sorting invariant

Definition 3.1 (Well-wrapped configuration)**.** A configuration of $\mathfrak{C}(G)$ is *well-wrapped* when it is well-sorted in the grammar of Construction 6.6 — in particular, every continuation occupies a slot of sort $\mathbb{T}$, so every redex occurrence lies inside a wrapper $\{\cdot\}_s$.

Lemma 3.2 (Subject reduction for wrapping)**.** *Well-wrapping is preserved by* (R1)–(R3)*.*

Proof sketch. Each rule replaces a wrapped redex by $K'(C'_{\mathrm{p}}, C'_{\mathrm{e}})$ with $(C'_{\mathrm{p}}, C'_{\mathrm{e}}) = \mathsf{compute}(\dots)$. Since $\mathsf{compute} : \cdots \to \mathbb{T} \times \mathbb{T}$ and K' packages terms of sort $\mathbb{T}$, the residual is well-sorted; the consumed cell leaves a well-sorted stack. Redexes elsewhere are untouched. Hence the result is well-wrapped. □

Corollary 3.3 (No leak, by construction)**.** *In a well-wrapped configuration no redex can fire without being unwrapped, i.e. without consuming a token. There is no dynamic wrapping operation; the cost-accounting steps only ever unwrap.*

The contrast with the earlier formulation is the substance of this revision.

Remark 3.4 (Eager vs. lazy metering)**.** An alternative discipline re-wraps the contractum at contraction time, traversing it to sign each exposed redex — charging *eagerly* for every redex the step exposes. Wrapping by construction charges *lazily*: each redex is born wrapped and is charged only when actually forced. Lazy metering pays for work performed, not work merely exposed; redexes beneath a discarded binder are never charged. The token stack drains exactly in step with the reductions performed, which is also the operational reality of phlogiston consumption.

Remark 3.5 (Duplication needs no fresh signatures)**.** Because multiplicity is carried by the *stack* (linear token consumption) and not by key distinctness, copying a wrapped term $\{Q\}_s$ copies its key s but not its tokens. Each copy must be forced separately and so funded separately from the stack: n copies of an s-redex require n cells keyed to s. Shared keys with a linear token supply already make duplication cost, so no fresh-per-copy minting is required. Created redexes — where a substituted wrapped term lands in head position — are guarded for the same reason: the substituted material carries its own wrapper. Thus a single mechanism (wrapped continuations + linear stack) handles duplicated and created redexes alike, where the eager formulation needed a separate fresh-signature discipline.

4 Two monoids: space and time

The construction places two combination operators side by side, of different character. The interaction constructor K is *spatial*: it records what is in contact with what, and may carry equations — associative–commutative in rho (an unordered bag of processes), free in λ (a positional application spine). The cons operator $::$ of (1) is *temporal*: it records what is spent next, then next. It is a free monoid (a list), *never* commutative, because reduction is sequential even when interaction is parallel. Each forcing pops a cell; the order of popping is the order of steps. The stack is the time axis of the computation reified as a term: $\equiv$ leaves it invariant (spatial congruence consumes no time), $\longrightarrow$ pops it (a step is a tick), and its ordering is the arrow of the computation.

Proposition 4.1 (Stack consumption is the modulus, lazily realised)**.** *For a terminating reduction in $\mathfrak{C}(G)$, the number of cells consumed equals the number of forced redexes, which equals the number of cuts eliminated — including those introduced by duplication, each charged when forced. The consumed prefix of the temporal stack is therefore an exact operational modulus for the cut elimination, realised by running the reduction rather than by a separate traversal.*

Where eager metering pre-computes a bound by traversing each contractum, wrapping by construction lets the reduction *be* the bound extraction: the consumed stack is the realised cost, tight by construction and lazy. Evaluation is cut elimination; the token stack is the extracted modulus; metering at the granularity of forcing makes the two coincide.

5 Signature construction as a section of the quotient

Tokenisation derives its expressiveness from minting signatures from runtime data, so the construction adjoins a constructor turning terms into signatures. We isolate what the base theory must provide.

5.1 Quotient and section

Let $\mathcal{T}$ be the free (pre-congruence) syntax of the interacting sort and $\widehat{\mathcal{T}} = \mathcal{T}/\equiv$ its quotient by structural congruence. Two maps relate them, differing precisely in their relationship to $\equiv$:

- the **quotient** $\mathcal{T} \twoheadrightarrow \widehat{\mathcal{T}}$, *respecting* $\equiv$, invertible up to congruence — the structure a name constructor exposes when channels are taken up to $\equiv$ (in rho, $@P$ with $@P = @Q \iff P \equiv Q$). It is not a constructor the cost functor requires; it is the congruence the cut is gated modulo.
- a **section** $\mathsf{cf} : \widehat{\mathcal{T}} \to \mathcal{T}$, a choice of canonical representative per class. Composing with a collision-resistant digest gives the signature constructor $\# = \mathrm{digest} \circ \mathsf{cf} : \widehat{\mathcal{T}} \to \mathsf{Sig}$, a one-way commitment to a representative. It does *not* respect $\equiv$ — and must not, since a commitment identifying congruent-but-distinct representatives would be no commitment.

Sig carries a commutative monoid $(\mathsf{Sig}, *, ())$ compounding signatures, matching the compound-signature gating of (R2), (R3). Signatures never reduce: they are a rewrite-free sub-theory meeting the behavioural theory only at the matching guard. Hence the $\equiv$-incoherence of $\#$ is harmless for bisimulation, which factors as "P-bisimulation gated by signature-key matching."

Remark 5.1 (The two axes, disentangled)**.** In rho the section and the communication-quotient are the *same* operator $@\cdot$ viewed two ways, which is why the cost story there appeared to need "two axes of reflection." They are not two signature schemes. One axis ($@\cdot$ up to $\equiv$) is the communication quotient the LTS already carries; the other ($\# = \mathrm{digest} \circ \mathsf{cf}$) is the authentication section the cost functor adds. The signature is $\#$ alone; the functor needs both because metering tracks *what a process does* (quotient) and *who committed to it* (section).

5.2 The λ-calculus supplies a section without a channel constructor

The λ-calculus has no analogue of $@\cdot$. This is the sharp test, and it passes, because the functor never required a *channel* constructor; it required a *section*. For λ the congruence is α and a section is a canonical representative of each α-class: the de Bruijn encoding [17]. Thus $\#_\lambda = \mathrm{digest} \circ (\text{de Bruijn})$, and a signature commits to the de Bruijn form. The communication-quotient role that $@\cdot$ played in rho is, in λ, filled not by a constructor but by α-equality of redexes in the guard. Rho fused section and quotient; λ pulls them apart, and the construction is identical.

Definition 5.2 (Section-equipped)**.** An interactive GSLT is *section-equipped* when it comes with a computable section $\mathsf{cf} : \widehat{\mathcal{T}} \to \mathcal{T}$ of its $\equiv$-quotient (equivalently, a computable canonical-form function on the interacting sort).

6 The cost endofunctor

6.1 Continued interactive GSLTs

Definition 6.1 (Continued interactive GSLT)**.** A *continued interactive GSLT* is an interactive GSLT (an object of **iGSLT**) *equipped with* the following additional structure:

(i) a presentation of its dynamics in interaction-cut form (†), naming the constructors $K, K_\mathrm{p}, K_\mathrm{e}, K'$ and the partial contraction $\mathsf{compute}$;

(ii) a section (Definition 5.2), i.e. it is section-equipped; and

(iii) *wrappability*: the contraction $\mathsf{compute}$ is well-sorted as an operation on wrapped continuations, $\mathsf{compute} : \cdots \to \mathbb{T} \times \mathbb{T}$ — equivalently, $\mathsf{compute}$ is definable on metered thunks, mapping wrapped inputs to wrapped outputs.

We write **ciGSLT** for the category whose objects are continued interactive GSLTs and whose morphisms are theory maps that are **bisimulation-preserving** on the interacting sort and **quote-faithful**: injective on the reachable signature keys.

The adjective *continued* thus names a strict enrichment of *interactive*: clauses (i)–(iii) are data and conditions added on top of an **iGSLT**-object, not a restriction of it. Forgetting them recovers the underlying interactive GSLT.

Definition 6.2 (The forgetful functor)**.** Let $U : \mathbf{ciGSLT} \to \mathbf{iGSLT}$ send a continued interactive GSLT to its underlying interactive GSLT — discarding the metered-cut presentation, the section, and the wrappability witness — and act as the identity on the underlying theory map of a morphism.

Proposition 6.3 (The distinction is proper)**.** *U is faithful but neither full nor essentially surjective. Hence* **ciGSLT** *is a genuine, proper subcategory of* **iGSLT***: strictly more structure on objects, strictly fewer morphisms between them.*

Discussion. Faithful, not full. U forgets no information about a morphism's action, so it is faithful. It is not full because a **ciGSLT**-morphism must additionally be quote-faithful: an **iGSLT**-morphism that is bisimulation-preserving yet collapses two distinct signature keys is a map between the underlying objects with no lift to **ciGSLT**. (This refines the naive expectation that the inclusion be full: the section structure on objects induces a real constraint on morphisms, so the inclusion is faithful-not-full rather than full.)

Not essentially surjective. Not every interactive GSLT admits the added structure. Clause (i) requires the dynamics to factor as an interaction cut — contact K then contraction $\mathsf{compute}$ —

which a theory with non-local or unfactorable rewrites need not; clause (ii) requires a *computable* section of the $\equiv$-quotient, which a theory with undecidable structural congruence lacks; clause (iii) requires the contraction to preserve wrapping. An interactive GSLT failing any of these is an object of **iGSLT** with no preimage under U. □

Remark 6.4 (λ exhibits the distinction)**.** The λ-calculus is an object of **iGSLT** *as such*: terms, α, β. It becomes an object of **ciGSLT** only once one *chooses* the added structure — the de Bruijn section for (ii), the degenerate-K_{e} interaction-cut presentation of β for (i), substitution-on-thunks for (iii) (Section 8). The same underlying calculus thus sits at both levels, witnessing that "continued" is added structure and not a property read off the interactive GSLT alone. This is exactly the role λ plays as a control: each clause is a real question about it, with a real answer.

The morphism conditions are dual and jointly necessary. Bisimulation-preservation forbids distinguishing too much (behaviourally); quote-faithfulness forbids collapsing too much (representationally). A morphism merging two signatures would let a term signed by one drain a token minted for the other, so the target would gain transitions the source lacked. $\mathfrak{C}$ lives exactly where both are jointly satisfiable.

Remark 6.5 (Wrappability replaces traversability)**.** The condition (iii) is the wrapped-by-construction successor of the earlier "traversable contractum" requirement. We no longer need to walk a contractum to re-sign its exposed redexes; we need only that the contraction be a well-sorted operation on thunks. An opaque contraction that destroyed wrapping — returning bare redexes — would violate (iii) and is excluded, which is the same exclusion as a contraction whose cost could not be accounted.

6.2 The functor on objects

Construction 6.6 ($\mathfrak{C}$ on objects)**.** For $G = (\Sigma, E, R, K, K_{\mathrm{p}}, K_{\mathrm{e}}, K', \mathsf{compute}, \mathsf{cf}) \in$ **ciGSLT**, let $\mathfrak{C}(G)$ be the continued interactive GSLT with:

- **signature** Σ extended by Sig (with $\# = \mathrm{digest} \circ \mathsf{cf}$, monoid $*, (\,)$); the wrapped-term sort $\mathbb{T}$ with $\{\cdot\}_s$; the token-stack sort Stk with $::$ and the located-stack constructor $S(I, -) : \mathit{Surf} \times \mathsf{Stk} \to \mathsf{Stk}$ (Section 10); the polymorphic typing of K over wrapped-term/stack and stack/stack pairs; the re-sorting of every continuation slot of $K_{\mathrm{p}}, K_{\mathrm{e}}$ to $\mathbb{T}$; and the polymorphic typing of the *surface* slot of $K_{\mathrm{p}}, K_{\mathrm{e}}$ over a bare surface (Surf), a sealed surface ($\mathit{Surf} \times \mathsf{Sig}$, written $\{I\}_s$), and a degenerate (structurally carried) surface — so that R1 fills the slot with a bare surface and R2/R3 with a sealed one, as instances of one constructor;
- **equations** E with the commutative-monoid equations on $(\mathsf{Sig}, *, (\,))$ and the inertness of Sig and Stk under $\longrightarrow$;
- **rewrites** the gated family (R1)–(R3), with the *unchanged* contraction $\mathsf{compute}$ (now read at sort $\mathbb{T} \times \mathbb{T}$);
- **section** the evident lift of cf ignoring inert sorts.

Proposition 6.7 (Closure)**.** $\mathfrak{C}(G)$ *is again a continued interactive GSLT.*

Proof sketch. The gated rules are themselves in interaction-cut form: the outer polymorphic K is contact, the signed redex and the token stack are the two operands, and $\mathsf{compute}$ (with the stack tail) is the contraction. $\mathfrak{C}(G)$ is section-equipped by the lifted cf, and wrappable because $\mathsf{compute}$ preserves $\mathbb{T}$ by Lemma 3.2. All three clauses of Definition 6.1 hold. □

6.3 The functor on morphisms

Construction 6.8 ($\mathfrak{C}$ on morphisms)**.** For $f : G \to H$ in **ciGSLT**, let $\mathfrak{C}(f)$ act as f on the interacting sort and transport the adjoined structure along f: signatures by $\#_H \circ f$ on canonical forms, wrappers and stacks componentwise, the polymorphic K structurally.

Theorem 6.9 ($\mathfrak{C}$ is an endofunctor on **ciGSLT**)**.** $\mathfrak{C} : \mathbf{ciGSLT} \to \mathbf{ciGSLT}$ *preserves identities and composition, and* $\mathfrak{C}(f)$ *is a* **ciGSLT***-morphism whenever* f *is.*

Proof sketch. Object closure is Proposition 6.7; identity and composition preservation are immediate from the componentwise definition. By the factoring of bisimulation as "P-bisimulation gated by key matching," and since f is P-bisimulation-preserving and quote-faithful, $\mathfrak{C}(f)$ preserves a gated transition iff f preserves the underlying P-transition and respects the key match — both hold. Quote-faithfulness transports along the injective $\#_H$. □

7 Graded adequacy

The token stack grades transitions of $\mathfrak{C}(G)$ by the signature monoid $(\mathsf{Sig}, *, (\,))$: a forced step is labelled by the signature(s) it consumes. Applying OSLF — which auto-generates a Hennessy–Milner logic [10] from a GSLT with an adequacy theorem, in the lineage of the namespace logic for the rho calculus [2] — to the graded transition system of $\mathfrak{C}(G)$ yields a *graded* Hennessy–Milner logic with modalities $\langle a \rangle_s$ indexed by the consumed signature.

Theorem 7.1 (Graded adequacy, schematic)**.** *For* $\mathfrak{C}(G)$ *the graded Hennessy–Milner logic generated by OSLF is adequate for quote-faithful bisimulation: two configurations satisfy the same graded-HML formulae iff they are quote-faithfully bisimilar.*

This also explains the throughput improvement mechanically. In the translation presentation, fuel acquisition was an encoded multi-step protocol on auxiliary channels; under $\mathfrak{C}$ it is a single primitive graded transition — the grading absorbs the sub-protocol into the step relation (*protocol fusion*). With wrapping by construction the fusion is total: there is no re-wrapping sub-step either, only forcing. Together with Proposition 4.1, phlogiston is the conserved grade, invariant under $\equiv$ (signatures inert), strictly consumed along $\longrightarrow$ (forcing drains the temporal stack), in the order recorded by $::$.

8 Anchoring instances across the migration spectrum

The construction applies uniformly across calculi whose interaction cut carries no information migration, migration by binding, or migration by spatial restructuring. We work several instances, ordered roughly by increasing migration. Table 1 summarises the interaction surfaces, continuations, and contraction for each.

8.1 CCS: the paradigmatic instance, null migration

CCS [7] is the paradigmatic continued interactive GSLT precisely because it exhibits the interaction cut with binding and migration stripped away. Instantiate: $K = |$ (associative–commutative); the introductions are prefixes $a.P$ and $\bar{a}.Q$ with interaction surfaces $I = a$, $J = \bar{a}$ and continuations $C_{\mathrm{p}} = P$, $C_{\mathrm{e}} = Q$; the surface match $\mathsf{near}(a, \bar{a})$ is complementarity; and pseudo-application carries no data, so $\mathsf{compute}$ degenerates to pure release, $C'_{\mathrm{p}} = P$, $C'_{\mathrm{e}} = Q$. The gated rule (R1) reads

$$\{a.\{P\}_{s_{\mathrm{p}}} \mid \bar{a}.\{Q\}_{s_{\mathrm{e}}}\}_s \mid s :: S \longrightarrow \{P\}_{s_{\mathrm{p}}} \mid \{Q\}_{s_{\mathrm{e}}} \mid S.$$

Instance	**Surfaces** I, J	**Prog. cont.** C_{p}	**Env. cont.** C_{e}	**Contraction** compute
CCS	$a,\ \bar{a}$ (complementary)	P	Q	pure release: $P \mid Q$ (no migration)
rho / π (synchronous)	$x,\ x$ (name-equal)	$\lambda y.P$	$[Q_1, Q_2]$ (sent + cont.)	pseudo-app: $P\{Q_1/y\} \mid Q_2$ (migration by binding)
rho / π (asynchronous)	$x,\ x$ (name-equal)	$\lambda y.P$	$[Q, -]$ (no cont.)	pseudo-app: $P\{Q/y\}$ (binding, no release)
λ	$x,\ -$ (K_{e} degenerate)	$\lambda x.M$	N (no residual)	application: $M\{N/x\}$ (binding, no release)
Ambient calculus	ambient name n (open n vs. $n[\cdot]$)	P	Q	boundary dissolve/move (spatial restructuring)
Interaction categories	shared interface (agreement)	process	process	interface composition (no migration)

Table 1: Anchoring instances across the migration spectrum. A surface entry of $-$ denotes a nominally degenerate surface, carried structurally by the rigidity of K (Section 10) rather than absent; an environment-continuation entry of $-$ denotes a degenerate (absent) continuation, which is what distinguishes asynchronous from synchronous output. Four migration modes appear: *no migration* (CCS, interaction categories), *binding* (rho, π, λ), and *spatial restructuring* (ambient calculus), where the contraction relocates or dissolves an ambient boundary rather than substituting.

No substitution occurs — nothing migrates from environment to program — yet the construction goes through unchanged: the continuations are wrapped thunks, forcing drains a token, and the AC structure of $\mid$ supplies the split cases. That CCS, with no binding anywhere, is a clean continued interactive GSLT is the evidence that binding is incidental to the construction and migration is optional.

8.2 Communication (rho, π): migration by binding

Instantiate at the rho or π calculus: $K = \mid$ (associative–commutative); $K_{\mathrm{p}} =$ for with surface $I = x$ and program continuation $C_{\mathrm{p}} = \lambda y.T = \{P\}_{s_{\mathrm{p}}}$ (under the abstraction on y); $K_{\mathrm{e}} =$! with surface $J = x$. The environment continuation is where synchronous and asynchronous output differ: synchronous output is the pair $C_{\mathrm{e}} = [\,Q_1, Q_2\,]$ of a sent process Q_1 and a continuation Q_2 (the sender resumes as Q_2 once received), whereas asynchronous output is $C_{\mathrm{e}} = [\,Q, -\,]$ with a degenerate second slot — the sender does not wait, so there is no continuation. Asynchrony is thus a degenerate environment continuation, the temporal counterpart of λ's degenerate environment surface. The

contraction compute is pseudo-application, i.e. semantic substitution; cf a normal form for the |-congruence. The surfaces $I = x = J$ match by name-equality, and pseudo-application migrates the sent process into the program continuation through y. For synchronous output, rule (R1) reads

$$\{\mathrm{for}(y \leftarrow x)\, T \mid x!(U)\}_s \mid s :: S \longrightarrow T\{@U/y\} \mid S,$$

and since $T = \{P\}_{s_\mathrm{p}}$ and $U = \{Q\}_{s_\mathrm{e}}$, the residual $T\{@U/y\} = \{P\{@\{Q\}_{s_\mathrm{e}}/y\}\}_{s_\mathrm{p}}$ is again a wrapped thunk, to be forced later by a token keyed to s_p. No re-wrapping occurs; the AC structure of | supplies the split-redex and split/combined-token cases (R2), (R3) up to $\equiv$, recovering the earlier five-rule lattice. Against CCS, the only difference is that pseudo-application here has a non-trivial substitution half; rho keeps a process in the sending position throughout, exchanged as its quote and recovered by drop.

8.3 β-reduction: migration, no environment residual

Instantiate at the λ-calculus: $K = \mathrm{App}$ (free, no equations); $K_\mathrm{p} = \lambda$ with surface $I = x$ and wrapped body $C_\mathrm{p} = \{M\}_{s_\mathrm{p}}$. The argument has a nominally degenerate surface and no residual, so K_e is *degenerate*: the match condition is carried structurally by the rigid App rather than by an explicit J (Section 10), and the argument is its own wrapped environment continuation $C_\mathrm{e} = \{N\}_{s_\mathrm{e}}$. Pseudo-application is then ordinary application — substitution with empty release, $\mathsf{compute}(x, -, \{M\}_{s_\mathrm{p}}, \{N\}_{s_\mathrm{e}}) = (\{M\}_{s_\mathrm{p}}\{\{N\}_{s_\mathrm{e}}/x\},\ ())$. Rule (R1) reads

$$\mathrm{App}\big(\{\mathrm{App}(\lambda x.\{M\}_{s_\mathrm{p}},\ \{N\}_{s_\mathrm{e}})\}_s,\ s :: S\big) \longrightarrow \mathrm{App}\big(\{M\}_{s_\mathrm{p}}\{\{N\}_{s_\mathrm{e}}/x\},\ S\big),$$

with $s = \#_\lambda$ of the de Bruijn form of the redex. Every copy of $\{N\}_{s_\mathrm{e}}$ that substitution places into M carries its wrapper, so duplicated redexes are guarded; and where x stood in head position, the landed $\{N\}_{s_\mathrm{e}}$ guards the newly created application — both by construction, with no traversal (Remark 3.5). Token supply is *not* function application; the temptation to write $(\{\mathrm{App}(\lambda x.M, N)\}_s\ s)$ feeds the key as a value and gates nothing. The token sits at the contact site and is consumed by forcing.

8.4 Ambient calculus: migration by spatial restructuring

The mobile ambient calculus [19] exhibits a contraction of a different character: not substitution but the restructuring of a spatial hierarchy. Instantiate $K = |$ (associative–commutative); for the open reduction $\mathrm{open}\, n.P \mid n[Q] \to P \mid Q$, the introductions are the capability $\mathrm{open}\, n.P$ and the ambient $n[Q]$, with interaction surfaces the ambient name n on each side (matched by near as name-equality of the capability against the ambient) and continuations $C_\mathrm{p} = P$, $C_\mathrm{e} = Q$. The contraction dissolves the boundary of n, releasing Q into the context; for the in and out capabilities it instead relocates a subtree across a boundary. In every case compute is a *spatial restructuring* — it changes which ambients are nested within which — rather than a substitution: no datum migrates through a binder, yet the configuration's nesting, and hence what is near what, is rearranged. The gated rule wraps and meters exactly as before, with the boundary operation as the contraction. The ambient calculus is therefore the cleanest instance in which the contraction is not substitution but a tree rewrite, and it is the one whose dynamics most directly realises the spatial reading of Section 13: a step literally moves information through space.

8.5 Interaction categories: migration as interface composition

The interaction categories of Abramsky, Gay, and Nagarajan [8] present interaction as composition of processes along a shared typed interface. In the present terms the interaction surfaces I, J are the

matching interface — the surface match $\mathsf{near}(I, J)$ is interface agreement — and the contraction is composition rather than substitution, with no binding. They occupy a distinct point on the spectrum from CCS: still binding-free, but with compute a genuine combination of the two continuations along their shared boundary rather than a pure release. That they too are continued interactive GSLTs shows the construction is indifferent to whether migration is by substitution, by spatial restructuring, by interface composition, or absent entirely; what it requires is the interaction-cut shape, a section, and a wrappable contraction.

9 The cost monad and two adjunctions

Three further facts organise the construction: iterated cost accounting flattens, so $\mathfrak{C}$ carries a monad structure; and that monad relates to its base through two different adjunctions — one structural, one computational — whose embeddings have opposite characteristics.

9.1 Flattening: $\mathfrak{C}$ is a monad

Applying the construction twice yields $\mathfrak{C}^2(G) = \mathfrak{C}(\mathfrak{C}(G))$: a calculus that meters the metering of G. Its signatures are signatures of signed terms, its stacks are stacks of stacks, and its wrappers nest as $\{\{P\}_{s_2}\}_{s_1}$. There is a canonical way to collapse the two layers into one, and — tellingly — it is assembled entirely from the two monoids already present in the construction.

Construction 9.1 (Unit and multiplication)**.** Define the *unit* $\eta_G : G \to \mathfrak{C}(G)$ to wrap each term with the unit signature and present it against the freely available unit token,

$$\eta_G(P) \;=\; \{P\}_{(\,)}.$$

Since $(\,)$ is the unit of $(\mathsf{Sig}, *, (\,))$, an $(\,)$-keyed redex fires without net resource; thus η_G embeds G as the *cost-free fragment* of $\mathfrak{C}(G)$ — the metered calculus restricted to trivially-funded interactions. (This is consistent with the source/image discipline of Section 3: η is precisely the wrapping that installs apparatus at zero charge.)

Define the *multiplication* $\mu_G : \mathfrak{C}^2(G) \to \mathfrak{C}(G)$ to flatten the two layers by

(i) **multiplying nested signatures** in Sig, $\{\{P\}_{s_2}\}_{s_1} \mapsto \{P\}_{\,s_1 * s_2}$; and

(ii) **concatenating the stack-of-stacks** into a single stack — the free-monoid (list) multiplication on Stk.

The outer account is thereby merged into the inner: the grade of the flattened redex is the product of the two grades, and its temporal stack is the concatenation, in the order the two layers would have drained them.

Proposition 9.2 (The cost monad)**.** $(\mathfrak{C}, \eta, \mu)$ *is a monad [21] on* **ciGSLT**. *Its laws descend from the laws of the two constituent monoids: the unit laws from* $s * (\,) = s = (\,) * s$ *together with the singleton/empty laws of the list monad, and associativity from associativity of* $*$ *and of list concatenation.*

Proof sketch. η and μ are **ciGSLT**-morphisms. Each is quote-faithful, since $*$ is injective in each argument up to the collision-resistance of the digest, and bisimulation-preserving, since flattening neither creates nor destroys gated transitions — it relabels each by the product grade and consolidates the (spatially commutative, temporally sequential) consumption order consistently. Naturality is the componentwise action on the adjoined sorts. The monad identities $\mu \circ \mathfrak{C}\eta = \mathrm{id} = \mu \circ \eta_{\mathfrak{C}}$ reduce to the unit laws of $*$ and of concatenation; $\mu \circ \mathfrak{C}\mu = \mu \circ \mu_{\mathfrak{C}}$ reduces to their associativity. □

Remark 9.3 (A genuine, non-idempotent resource monad)**.** $\mathfrak{C}$ is not idempotent: μ_G is no isomorphism, because concatenating two stacks forgets the boundary between them and multiplying two grades forgets the factorisation. Flattening is a true *merge* of two resource accounts, not the collapse of a redundant copy. This is the expected signature of a resource monad — re-metering a metered calculus yields a finer account, which μ then consolidates — and it distinguishes $\mathfrak{C}$ from idempotent "closure" monads.

The Eilenberg–Moore algebras of $\mathfrak{C}$ are continued interactive GSLTs equipped with a coherent *discharge* map $\alpha : \mathfrak{C}(A) \to A$ interpreting tokens — a calculus that knows how to "pay" — and the Kleisli category is that of cost-accounted translations between calculi. The structural adjunction below is the free resolution generating the monad; the Turing-complete adjunction is a separate phenomenon.

9.2 Adjunction I: install ⊣ forget

Let **Cost-ciGSLT** be the category of *cost-accounted* continued interactive GSLTs: objects carry the installed apparatus (Sig, Stk, the wrapper, the gated family), morphisms preserve it. A forgetful functor $\mathsf{Forget} : \mathbf{Cost\text{-}ciGSLT} \to \mathbf{ciGSLT}$ strips the apparatus, returning the underlying un-metered calculus; a free functor $\mathsf{Free} : \mathbf{ciGSLT} \to \mathbf{Cost\text{-}ciGSLT}$ installs it — exactly Construction 6.6.

Proposition 9.4 (Free–forgetful resolution)**.** $\mathsf{Free} \dashv \mathsf{Forget}$*, with unit* η *of Construction 9.1, and the induced monad* $\mathsf{Forget} \circ \mathsf{Free}$ *on* **ciGSLT** *is* $\mathfrak{C}$*.*

The embedding is *structural and strict*: Free adjoins sorts and rules, Forget erases them on the nose. Crucially, forgetting is *not* behaviour-preserving in the metering sense — erasing the apparatus removes the gating, so $\mathsf{Forget}\,\mathsf{Free}\,G$ runs G's interactions *without friction*. The structural resolution answers "what apparatus was added?" and lets us remove it; it does not claim the metered and un-metered dynamics agree. Its character is **structure-preserving, behaviour-altering**.

9.3 Adjunction II: internalise ⊣ include, over a universal base

Restrict to the full subcategory $\mathbf{ciGSLT}_{\mathrm{TC}} \hookrightarrow \mathbf{ciGSLT}$ of *Turing-complete* continued interactive GSLTs. For such a G, the entire metering apparatus of $\mathfrak{C}(G)$ — signature equality, stack maintenance, gating — is computable, hence *encodable in* G *itself*. This yields an *internalisation*

$$\mathcal{I}_G : \mathfrak{C}(G) \longrightarrow G,$$

realising the cost-accounted semantics inside the un-metered but universal base by simulation: tokens become encoded data, the gated rules become an interpreter loop, and each forced step is simulated by a finite run of G.

Proposition 9.5 (Internalisation as an adjoint retraction)**.** *For* $G \in \mathbf{ciGSLT}_{\mathrm{TC}}$ *the internalisation satisfies* $\mathcal{I}_G \circ \eta_G \approx \mathrm{id}_G$ *up to weak bisimulation, exhibiting* $\mathfrak{C}(G)$ *as a behavioural retract of* G*. In the bicategory of continued interactive GSLTs, simulations, and simulation transformations,* $\eta_G \dashv \mathcal{I}_G$ *is an adjoint retraction: the unit* η_G *is an (iso-up-to-bisimulation) section and the counit is the collapsing simulation* $\eta_G \circ \mathcal{I}_G \Rightarrow \mathrm{id}_{\mathfrak{C}(G)}$*.*

Proof sketch. Turing completeness of G gives a computable encoding of the finite data and decidable guards of $\mathfrak{C}(G)$; standard interpreter constructions make $\mathcal{I}_G$ a weak simulation that reflects every gated transition. Composing with η_G (which installs apparatus at unit cost) returns G's dynamics up to the administrative reductions of the interpreter, i.e. up to weak bisimulation. The triangle identities hold up to the 2-cells witnessing these weak bisimulations, which is the content of an adjunction internal to the simulation bicategory. □

The embedding is the mirror image of Adjunction I. It is *behaviour-preserving* — the simulation faithfully reproduces every gated step — but *structure-encoding rather than structure-preserving*: the clean sorts $\mathsf{Sig}, \mathsf{Stk}$ dissolve into encoded data, and the agreement holds only up to weak bisimulation, since the interpreter interposes administrative reductions. Its character is **behaviour-preserving, structure-dissolving**, exactly opposite to Adjunction I.

Remark 9.6 (Two senses of conservativity, and where they hold)**.** The two adjunctions express two senses in which cost accounting is conservative over its base. Structurally (Adjunction I, available for every object) the apparatus is a detachable layer. Computationally (Adjunction II) the apparatus is already expressible, so metering is a definitional extension up to weak bisimulation — but this holds *only* when the base is Turing complete. The second is therefore a genuine property of the object: universal interactive theories are exactly those whose cost-accounted versions fold back into them, a distinction invisible to the purely structural resolution.

10 Capabilities: located authority and the theory of K

A cost-accounted calculus is a calculus in which the right to draw on a resource stack is itself an authority. We must therefore ask what confers that authority, and ensure the construction does not silently grant it — that it does not introduce *ambient authority* [11, 12], the antithesis of an object-capability discipline. The answer turns entirely on the equational theory of the interaction constructor K, and it is most cleanly stated through the interaction surface of Section 2. The surface — the part that must match for a cut to fire — can be carried in two ways. It may be carried *structurally*, by the rigidity of K: when K is *free* — carrying no equations at all — position is unforgeable and the context $K([\,], -)$ itself is the surface. Or it must be carried *nominally*, by an explicit I, J related by the nearness operator, which becomes necessary as soon as K carries *any* equation. The dividing line is freeness, not commutativity: an equation on K is a congruence step — it rewrites adjacency *without firing a cut*, consuming no token — so any equation lets a program manoeuvre itself next to a stack it does not structurally neighbour. Associativity re-brackets adjacency, a unit law inserts and deletes neighbours, idempotence copies them, and associative–commutativity dissolves position altogether; each forges position, and only a free K admits no such move. A nominally degenerate surface, as on λ's environment side, is thus not a missing surface but a structurally carried one. These are the two faces of a single notion, and which one a calculus uses is dictated by the equational theory of K: free, or not.

10.1 The structural surface: position as capability when K is free

When K carries no equations, position is unforgeable: nothing can drift into contact with a stack it does not already neighbour. The rigidity of K thereby *supplies* the interaction surface — the surface is not recorded by any explicit I, J but by the structural context. A capability to draw on a stack S is then literally that context

$$K([\,],\, S),$$

the right to occupy the hole adjacent to S — to be "next to the cookie jar." Because K is rigid, occupying that position is the whole of the authority: no other program can manoeuvre itself into the same position, so confinement is automatic and the capability is exactly the context. This is the λ-calculus regime, and it is the object-capability ideal: authority is structural, held by reference, and unforgeable by construction. This is also what the degenerate environment surface of β (Section 2) amounts to — not the absence of a surface but its structural provision by the rigid App.

10.2 The leak of ambient authority when K carries equations

The moment K carries any equation, position is no longer exclusive: an equation is a congruence step that changes adjacency without firing a cut, so it lets a program drift into contact with a stack it did not structurally neighbour. Associativity re-brackets adjacency, a unit law inserts or deletes neighbours, and idempotence copies them; associative–commutativity — the rho regime, $K = |$ — is the maximal case, dissolving position entirely into an unordered bag. Taking that maximal case as the witness: because $|$ is AC the configuration

$$K\big(K_{\mathrm{p}}^1(I^1, C_{\mathrm{p}}^1),\ K_{\mathrm{p}}^2(I^2, C_{\mathrm{p}}^2),\ K_{\mathrm{e}}(J, C_{\mathrm{e}})\big)$$

is well-formed, and *both* introductions $K_{\mathrm{p}}^1, K_{\mathrm{p}}^2$ sit adjacent to the eliminand. Mere adjacency can no longer be the capability, because adjacency has become cheap and non-exclusive: every program in the bag is "next to" every stack in it. Taking proximity as authority here is exactly ambient authority, and it is a leak — and the same leak opens, in milder form, under any nontrivial equation on K, since each is a way to rewrite a program into a position it did not structurally hold.

10.3 The nominal surface: the nearness operator when K is non-free

To recover an object-capability discipline under a non-free K — where the structural surface is unavailable because position is forgeable — we carry the surface nominally instead. The interaction surfaces I (of the receiving introduction) and J (of the eliminand) are the parts that must match, and we combine them with a nearness operator

$$\mathsf{near}(I, J)$$

which, when defined, returns the surface at which they meet; interaction is gated on its being defined — the gated rules of Section 3 fire only when $\mathsf{near}(I, J)$ yields a surface, and that surface locates the purse. Interaction is then licensed not by position but by the matchability of the two surfaces; I and J become a *proxy for capability*. The operator specialises across the spectrum: it is defined by complementarity of actions in CCS, name-equality of subjects in rho and π, and interface agreement in the interaction categories.

The two carriers are now one notion seen twice. In the free case the surface is the context $K([\,], S)$ and near is trivially defined — being in the hole *is* being near. In the non-free case the structural context is no longer exclusive, so the same surface must be carried nominally by I, J and supplied by near. The match condition has not changed; only its carrier has, from the geometry of K to an explicit operator. Equivalently: a free K supplies the surface structurally and needs no gate, while any equation on K loosens the structural surface — dissolving it entirely in the AC case — and so forces it to be reintroduced nominally to avoid ambient authority.

Remark 10.1 (Nearness bounds but does not eliminate competition)**.** $\mathsf{near}(I, J)$ does not by itself make access exclusive: two receivers I^1, I^2 may both be near the same J, so competition for a stack remains possible. What the operator guarantees is that only holders of a *matching surface* may compete at all — it replaces "anyone adjacent" with "anyone holding a matching surface," which is precisely the recovery of capability discipline. Exclusivity, where required, is an additional matter for the type discipline of Section 11.

10.4 Located resource stacks

The nearness analysis motivates refining the stack constructor itself. Rather than a bare list of signatures floating in the AC bag, a stack is *located* at an interaction surface:

$$S(I, s :: S'),$$

a stack indexed by the surface I that may draw on it. The cookie jar now carries a label naming which hands may reach in. Locating the stack turns "reachable by whatever is adjacent" into "reachable by a matching surface," so authority over a resource is named in the term rather than conferred by the geometry of the bag. This is the form on which the type discipline and the resource-sufficiency proofs of the next two sections operate; in the free regime the location is implicit in the context $K([\,], S)$, while under any non-free K it is the explicit carrier of authority.

Remark 10.2 (Naming coincidences in the rho instance)**.** In rho the receiver's surface is $I \sim (y, x)$ and the eliminand's is $J \sim x$, so $\mathsf{near}(I, J)$ partly coincides with channel-name matching: the channel already does surface-like work. The general construction separates the two notions — name matching selects *which token* is spent (the signature guard), while near governs *who may approach* the stack at all. Rho fuses them on the channel name, as it fused quotient and section (Section 5); the located form keeps them distinct.

11 A type discipline for token usage

Linearity of tokens should not be imposed by fiat in the operational semantics, for some token types ought to be *copyable* — freely duplicable authorisations — while others are *linear* — spent exactly once. Rather than hardwire a single choice, we let the discipline be *checked*: a programmer asserts the usage discipline a term respects, and the assertion is verified against a type system. The type system is not new machinery; it is the logic OSLF already generates.

11.1 OSLF as a checkable discipline

Running OSLF on the cost-accounted theory $\mathfrak{C}(G)$ — not on the bare G — yields a Hennessy–Milner-style logic for the metered calculus, adequate for quote-faithful bisimulation (Theorem 7.1). We read this logic as a type system: a term is *typed* by checking that it satisfies a formula expressing the intended token discipline. The discipline is opt-in and per-term: the calculus does not force linearity, but a programmer who requires it writes a term and checks it against the linear formula. Copyable tokens are simply those whose types impose no single-use constraint.

11.2 Spatial and modal connectives suffice

Two connective families generated by OSLF are enough to express a variety of token disciplines.

Spatial types $K(\varphi_1, \varphi_2)$ read the interaction constructor as a separating-style connective: a term satisfies $K(\varphi_1, \varphi_2)$ when it splits across K into a part satisfying φ_1 and a part satisfying φ_2. This is the bookkeeping of *duplication and partition* — whether token-bearing sub-terms are shared or held disjointly, and how many copies are present. When K is AC this is a genuine separating conjunction in the sense of spatial logic; when K is free it is a positional split. The discipline of linearity is thus, in its static aspect, spatial. The reading of an interaction constructor as a separating conjunction is that of spatial and separation logics [15, 13, 14].

Modal types $\langle K(\dots)\rangle\varphi$ are the graded modalities of Section 7, indexed by the interaction: a term satisfies $\langle K(\dots)\rangle\varphi$ when it can take a K-interaction — a draw on the stack — after which φ holds. This is the dynamics of *consumption*: that a spend is available and what the post-state looks like.

Combining the two expresses the usual lattice of disciplines. Writing ℓ for a token-bearing predicate, linearity is a spatial constraint forbidding a second copy ($\neg\, K(\ell, K(\ell, \top))$, no split exposing two live copies) together with a modal obligation that the single copy be spent; copyability drops

the spatial constraint; relevance keeps the modal obligation while permitting copies. The point is not a fixed type system but that the OSLF-generated logic of $\mathfrak{C}(G)$ is already expressive enough to host them.

11.3 Finiteness and location make checking tractable

Two properties of the located stack of Section 10 turn this from specifiable into statically decidable.

Finiteness: if every token stack is finitely bounded — of statically known, finite depth — the spatial connective accounts over a bounded budget, so "is there enough" and "is there at most one copy" are finite checks rather than reasoning about an unbounded resource.

Location: because each stack sits at a named surface $S(I, -)$ rather than floating in the bag, the analysis is local — consumption is reasoned about per surface, not globally over all adjacent competitors. Finiteness bounds the quantity; location bounds the scope. Together they make the spatial/modal discipline decidable, and they are exactly the refinements the capability analysis already forced.

12 Resource sufficiency: linear proofs and located purses

In the rho-specific development, the guarantee that a deployment carries enough phlogiston to run to completion was discharged by a *linear proof*: a derivation in a linear logic [16] witnessing that the available tokens suffice to cover every interaction the term will perform, checked once at deploy time. We ask whether this lifts to the general $\mathfrak{C}(G)$ setting, and whether the located stacks of Section 10 improve it.

12.1 Data-dependent interaction

A linear proof certifies a *fixed* consumption: it sums the tokens a term will draw and checks the sum against the supply. This is sound when the interactions a term performs are determinable statically. Under *data-dependent* interaction it is not immediate. If a received value selects a branch, or a name computed at run time determines a near match, then which interactions fire — and so how many tokens of which signatures are drawn — depends on the run, and there is no single fixed quantity for the proof to sum.

Two honest responses follow. For the data-*independent* fragment the rho-era guarantee lifts verbatim: the consumption is fixed and the linear proof bounds it. For the data-*dependent* fragment the proof must either become *dependent* — parameterised by the data, with the branch condition carried in the modal type $\langle K(\dots)\rangle\varphi$ of Section 11, so that each branch discharges its own linear obligation — or fall back to a *conservative* static bound, the worst case over branches, which is sound but may over-charge. The linear proof therefore suffices unconditionally only for the static fragment; data-dependence requires either dependency in the proof or a conservative envelope.

12.2 Located purses as an optimisation

The located stacks $S(I, -)$ — "purses" parked at interaction surfaces — optimise the linear proof regardless of which response above is taken. Instead of one global linear derivation accounting for the whole term's draw against a single shared pool, locating the resources decomposes the obligation into *one local linear proof per purse*, each bounding only the consumption at its surface. The spatial connective $K(\varphi_1, \varphi_2)$ is precisely the proof-theoretic seam along which the global proof factors: a separating split of the term induces a split of its resource obligation into independent conjuncts.

Proposition 12.1 (Local sufficiency composes)**.** *If each purse $S(I, -)$ admits a local linear proof that its supply covers the consumption at surface I, and the surfaces partition the term's interactions, then the conjunction of the local proofs is a proof of global resource sufficiency. The global linear derivation factors as the separating conjunction of the per-purse derivations.*

This is the lifted and sharpened form of the rho-era guarantee. Beyond mere economy of proof, location *quarantines* the hard case: if data-dependence is confined to a particular surface, only that purse's sub-proof needs the dependent or conservative treatment, while every other purse retains a simple static linear proof. Locality thus does double duty — it makes the sufficiency proof compositional, and it contains the data-dependent reasoning to the surfaces that actually require it.

13 Conclusion

The cost-accounting move of the rho calculus is an instance of a generic endofunctor on the category **ciGSLT** of *continued* interactive GSLTs: interactive theories presented as an interaction cut (contact K, two introductions $K_{\mathrm{p}}(I, C_{\mathrm{p}})$, $K_{\mathrm{e}}(J, C_{\mathrm{e}})$ meeting along matching interaction surfaces I, J, contraction compute), equipped with a computable section of their structural-congruence quotient, and with continuation slots sorted as *wrapped terms by construction*. The functor $\mathfrak{C}$ signs terms, adjoins a token stack, makes the contact constructor K polymorphic, and replaces the cut by a gated family. Its soundness is a sorting invariant: every redex is born inside a wrapper, reduction preserves wrapping (Lemma 3.2), so no computation proceeds for free and the contraction need not be lifted — the cost-accounting steps only ever unwrap. Metering is thereby lazy, paying for work performed rather than exposed, and the temporal token stack is consumed exactly in step with reduction, realising the modulus of cut elimination as the run length. The cons operator is extension in time, dual to the spatial extension of K; OSLF lifts $\mathfrak{C}(G)$ to a graded Hennessy–Milner logic with adequacy "graded-HML equivalence equals quote-faithful bisimulation." The contraction is Milner's pseudo-application, and *binding is one mode of information migration* under it, not a feature of the cut: CCS instantiates the same shape with no migration and is in that sense paradigmatic, the interaction categories of Abramsky, Gay, and Nagarajan migrate by interface composition, and the rho and λ calculi migrate by substitution. The λ-calculus, with K and the section visibly distinct and K_{e} degenerate, shows the construction requires a section — de Bruijn canonicalisation — not a channel constructor; rho's apparent need for two axes of reflection was the self-overlap of one operator serving as both quotient and section.

Finally, the construction is deliberately sited one rung below the ambient category: $\mathfrak{C}$ is an endofunctor on **ciGSLT**, not on **iGSLT**, and the forgetful functor U : **ciGSLT** $\to$ **iGSLT** (Definition 6.2, Proposition 6.3) records that "continued" is a proper structural enrichment of "interactive." The content of the note is therefore as much about *where* cost accounting lives as about what it does: an interactive GSLT is a setting for interaction; a continued interactive GSLT is one whose interaction is presented as a meterable interaction cut, and it is precisely on these that the cost endofunctor exists — indeed, as a monad (Section 9), since iterated metering flattens. Its two resolutions sharpen the picture: the structural free–forgetful adjunction shows the apparatus is detachable, while over a Turing-complete base the internalisation adjunction shows it is already expressible, so that cost accounting is a definitional extension up to weak bisimulation. That second adjunction holds only for universal bases, and so quietly begins to sort interactive theories by a structural property — a thread we leave for the sequel. The migration spectrum of Section 8 already includes a calculus whose contraction is spatial restructuring rather than substitution — the mobile ambient calculus [19] — confirming that the present phenomena depend not on substitution but on any wrappable contraction. A natural next step is Cardelli's brane calculi [20], which generalise

ambients by placing the computation on the membranes (boundaries) rather than inside them, with interaction by membrane fusion and fission; they are a further family of continued interactive GSLTs whose contraction is spatial restructuring, and the proving ground for how the construction behaves when the active sites are themselves the boundaries that the restructuring moves.

Three further refinements turn the metered calculus into a usable resource discipline, and each is governed by the same structural feature — the equational theory of K — that organised the construction. The authority to draw on a resource is a capability, and whether the construction grants it ambiently depends on whether K is free: a free K makes position an unforgeable capability $K([\,], S)$, while *any* equation on K forges position and so demands an explicit nearness operator $\mathsf{near}(I, J)$ and a stack *located* at its surface, $S(I, -)$, to keep authority from becoming ambient (the AC theory of rho being the maximal case). On the metered calculus, OSLF generates not merely a behavioural logic but an opt-in type discipline: spatial types $K(\varphi_1, \varphi_2)$ and modal types $\langle K(\dots)\rangle\varphi$ together express linear, copyable, and relevant token disciplines, made statically decidable by the finiteness and location of stacks. And the rho-era guarantee of resource sufficiency by linear proof lifts: verbatim for the data-independent fragment, with dependent or conservative treatment for data-dependent interaction, and — crucially — factored by located purses into a separating conjunction of per-surface proofs, which both makes sufficiency compositional and quarantines the data-dependent case to the surfaces that require it. Authority, usage, and sufficiency thus form three layers on the one cost-accounted object: who may reach a stack, how a holder may spend what they draw, and whether the supply suffices — each a checkable discipline, each turning on the structure of K.

The interaction cut (†) is binary throughout this note: two introductions $K_{\mathrm{p}}(I, C_{\mathrm{p}})$ and $K_{\mathrm{e}}(J, C_{\mathrm{e}})$ meet along a single pair of surfaces. The process calculi on which the construction is modelled are, however, typically used with *joins* — a single program-side introduction awaiting on several surfaces at once, $K_{\mathrm{p}}(I_1, \dots, I_n;\, C_{\mathrm{p}})$, against n environment-side introductions, firing only when all n rendezvous succeed simultaneously: an n-ary cut whose contraction substitutes for all n bindings at once. We have kept the present development binary by design, and record here that doing so does not foreclose the n-ary case but leaves room for it. The pivotal observation is that sealing each surface in its own wrapper splits the n-ary program introduction into n binary-shaped introductions, one per surface; the atomicity of the join — all n fire together or none does — is then carried not by the syntactic unity of the introduction but by the indivisibility of a single compound funding cell keyed to the $*$-product of the n signatures. So long as that compound token may not be partially consumed, no constituent rendezvous can fire alone, and the binary apparatus already in hand — wrapping by construction and the located stack $S(I, -)$ — supplies the materials. What an n-ary treatment must add is exactly two things, and neither revises the binary construction: a *variadic* contraction $\mathsf{compute}$, sending n surface pairs to one simultaneous contractum in place of the binary $\mathsf{compute}$ of (†); and a stance on how permissive near may be across the field of candidate surface pairs. If near is constrained so that each program surface matches a unique environment surface, the join's outcome is forced and the binary accounting lifts verbatim, the consumed signature multiset being matching-invariant. If near is genuinely permissive — the reading under which distant surfaces may yet interact provided a located stack funds the meeting — the join fires on a *minimum-cost perfect matching* of its candidate pairs, and the located surfaces of Section 10 acquire a metric reading we find suggestive but do not pursue here. Either way the choice is downstream of the binary endofunctor rather than a modification of it; we leave the n-ary cost monad to the sequel.

Some correspondences to physical theories

We close with two correspondences to physical theories that, while not theorems of this note, are detailed enough to invite investigation. Neither is claimed as more than a curiosity; we record them in the spirit of the Curry–Howard correspondence, which began as a noticed resemblance — proofs look like programs — and proved foundational. Both rest on the same substrate: the two monoids of Section 4, K spatial (recording what is near what) and $::$ temporal (recording what is spent next), with cut elimination as the dynamics.

Noticing physical structure in the dynamics of cut elimination is not new, and we situate what follows in an older programme rather than claim to open the register. Girard's *Geometry of Interaction* took cut elimination as a dynamics — proof normalisation as the resolution of a feedback equation, the execution formula — and pursued it through operator algebras and C^*-algebras precisely in search of such correspondences [22, 23]. The categorical Geometry of Interaction of Abramsky, Jagadeesan, Haghverdi, and Scott [24, 25, 26] abstracted that dynamics into traced monoidal structure — the last of these showing the operator-algebraic original is exactly captured categorically — and the line continues into the categorical quantum mechanics of Abramsky and Coecke [27], where the same interaction structure yields a semiring of scalars and a Born rule, the very apparatus the second reading below leans on. What we add is two specific readings keyed to the two monoids of *this* construction, offered in the same spirit.

The first correspondence is with *general relativity*. Read a K-tree as an initial distribution of matter and energy — the information carried by the program and environment continuations, arranged in space. Cut elimination evolves this distribution: each interaction consumes two introductions, migrates information through the contraction, and leaves a contractum whose redexes have a *new* adjacency. What is near what is therefore not fixed but *dynamical*, changing as the computation runs. The ambient calculus makes this vivid: there the contraction is spatial restructuring (Section 8), so a step *literally* relocates information in the spatial hierarchy, and the change in adjacency is not a side effect of substitution but the whole content of the step.

The shape of this dependence is the shape of Einstein's intuition. In general relativity the distribution of matter and energy determines the metric — what is near what — and the metric in turn governs how matter moves. Here the distribution of information in the K-tree determines the nearness structure (which cuts are enabled), and the nearness structure governs how information flows at the next step, which redistributes the information. The analog of the stress–energy source is the information distribution; the analog of the metric is the nearness structure; and cut elimination is the coupling between them, each step letting the distribution reshape the geometry and the geometry select the next redistribution.

The analogy sharpens, rather than dissolves, when object capabilities enter. Without an authority discipline, space (K) and time ($::$) are separable: nearness and consumption order are independent coordinates. But the located stack $S(I, -)$ of Section 10 ties a temporal resource — what may be spent next — to a spatial surface — who is near. A draw then requires *both* spatial proximity (a matching surface, $\mathsf{near}(I, J)$) and temporal availability (a token at the head of the stack), so the spatial and temporal structures can no longer be varied independently. Authority *entangles* them. What was a product of a space and a time becomes a single coupled structure gating interaction — a spacetime. That the same structural feature, the theory of K, governs both the geometry and its entanglement with time is the part of the correspondence we find most worth pursuing, and we leave its formal development — whether the nearness evolution obeys anything like a field equation — to future work.

The second correspondence is with *quantum field theory*, and it turns on what a token *carries*. Read a token distribution as a collection of fields over the surfaces: a token resident at a surface

is the strength of its field there, and its signature is an internal (flavour) index distinguishing one field from another. Since a surface may carry many tokens of different keys — many strengths at one point — a token distribution is a map $\phi : \mathit{Surf} \times \mathsf{Sig} \to R$ valued in a semiring R, with $\phi_s(I)$ the strength of mode s at surface I. The cost calculus developed above is the shadow of this field in a degenerate semiring: the Boolean semiring $(\{0,1\}, \vee, \wedge)$ recovers the bare nondeterministic transition system, and the tropical semiring $(\mathbb{R} \cup \{\infty\}, \min, +)$ recovers cost — in which, tellingly, the minimum-cost perfect matching that the surface-valued near already produced for joins is precisely the tropical reading of the field. Taking R to be the non-negative reals gives probabilistic interaction; taking it to be the complex numbers gives *amplitudes*, and with them interference, a quantum-field-theoretic dynamics in which the Born rule $|\cdot|^2$ is a separate extraction laid on top. One obstruction is worth naming, and is why we take this no further here: interference adds and *cancels* contributions to a shared outcome, whereas bisimulation — the equivalence on which this note is built — is a branching-time notion that forbids exactly such cancellation, so the genuinely quantum reading pulls toward a linear-time semantics, a reformulation rather than a relabelling.

Both correspondences are, in all likelihood, no more than coincidence: a syntactic construction resembles many things. We record them anyway because the precedent for taking such a resemblance seriously is a good one — indeed two. The Geometry of Interaction programme above already took the dynamics of cut elimination seriously as physical-mathematical structure; and the Curry–Howard correspondence was, at first, only the observation that proofs look like programs; it was offered modestly and turned out to organise a large part of logic and computation. Whether the present resemblances — cut elimination coupling information to nearness in the manner of a field equation, and token distributions as fields whose complex weights restore interference — are more than coincidence is exactly the question we find worth asking. We leave a serious treatment of both, where the physics is taken on its own terms, to a separate note.

Acknowledgments

The development of this note benefited from extended dialogue with Anthropic's Claude, which served as a sounding board throughout: testing the generality of the interaction-cut formulation, surfacing the distinction between the structural and nominal carriers of an interaction surface, helping articulate the wrapped-by-construction discipline and its soundness as a sorting invariant, and pressing on the object-capability and resource-sufficiency arguments. The ideas and their errors are the author's; Claude's contribution was the kind of attentive counterpoint one hopes for from a colleague.

References

[1] L. G. Meredith and M. Radestock. A reflective higher-order calculus. In M. Viroli, editor, *ETAPS 2005 Satellite Events*, Electronic Notes in Theoretical Computer Science, vol. 141(5), pp. 49–67. Elsevier, 2005.

[2] L. G. Meredith and M. Radestock. Namespace logic: A logic for a reflective higher-order calculus. In *Trustworthy Global Computing (TGC 2005)*, Lecture Notes in Computer Science, vol. 3705, pp. 353–369. Springer, 2005.

[3] M. Stay and L. G. Meredith. Representing operational semantics with enriched Lawvere theories. In *Proceedings of the Higher-Dimensional Rewriting and Applications workshop (HDRA)*, 2017. arXiv:1704.03080.

[4] J. C. Baez and C. Williams. Enriched Lawvere theories for operational semantics. *Applied Category Theory (ACT)*, 2019. arXiv:1905.05636.

[5] R. Milner. *Communicating and Mobile Systems: the π-Calculus*. Cambridge University Press, 1999.

[6] R. Milner. The polyadic π-calculus: a tutorial. In F. L. Bauer, W. Brauer, and H. Schwichtenberg, editors, *Logic and Algebra of Specification*, NATO ASI Series, vol. 94, pp. 203–246. Springer, 1993.

[7] R. Milner. *A Calculus of Communicating Systems*. Lecture Notes in Computer Science, vol. 92. Springer, 1980.

[8] S. Abramsky, S. J. Gay, and R. Nagarajan. Interaction categories and the foundations of typed concurrent programming. In M. Broy, editor, *Deductive Program Design: Proceedings of the 1994 Marktoberdorf Summer School*, NATO ASI Series, pp. 35–113. Springer, 1996.

[9] D. Sangiorgi and D. Walker. *The π-Calculus: A Theory of Mobile Processes*. Cambridge University Press, 2001.

[10] M. Hennessy and R. Milner. Algebraic laws for nondeterminism and concurrency. *Journal of the ACM*, 32(1):137–161, 1985.

[11] M. S. Miller, K.-P. Yee, and J. Shapiro. Capability myths demolished. Technical Report SRL2003-02, Systems Research Laboratory, Johns Hopkins University, 2003.

[12] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.

[13] P. W. O'Hearn, J. C. Reynolds, and H. Yang. Local reasoning about programs that alter data structures. In *Computer Science Logic (CSL 2001)*, Lecture Notes in Computer Science, vol. 2142, pp. 1–19. Springer, 2001.

[14] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS 2002)*, pp. 55–74. IEEE, 2002.

[15] L. Caires and L. Cardelli. A spatial logic for concurrency (part I). *Information and Computation*, 186(2):194–235, 2003.

[16] J.-Y. Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987.

[17] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. *Indagationes Mathematicae*, 34(5):381–392, 1972.

[18] B. C. Smith and J. des Rivières. Interim 3-LISP reference manual. Technical report, Xerox PARC, 1984.

[19] L. Cardelli and A. D. Gordon. Mobile ambients. *Theoretical Computer Science*, 240(1):177–213, 2000.

[20] L. Cardelli. Brane calculi — interactions of biological membranes. In V. Danos and V. Schächter, editors, *Computational Methods in Systems Biology (CMSB 2004)*, Lecture Notes in Computer Science, vol. 3082, pp. 257–278. Springer, 2005.

[21] S. Mac Lane. *Categories for the Working Mathematician*, 2nd edition. Graduate Texts in Mathematics, vol. 5. Springer, 1998.

[22] J.-Y. Girard. Towards a geometry of interaction. In J. W. Gray and A. Scedrov, editors, *Categories in Computer Science and Logic*, Contemporary Mathematics, vol. 92, pp. 69–108. American Mathematical Society, 1989.

[23] J.-Y. Girard. Geometry of interaction I: interpretation of system F. In R. Ferro et al., editors, *Logic Colloquium '88*, Studies in Logic and the Foundations of Mathematics, vol. 127, pp. 221–260. North-Holland, 1989.

[24] S. Abramsky and R. Jagadeesan. New foundations for the geometry of interaction. *Information and Computation*, 111(1):53–119, 1994.

[25] S. Abramsky, E. Haghverdi, and P. Scott. Geometry of interaction and linear combinatory algebras. *Mathematical Structures in Computer Science*, 12(5):625–665, 2002.

[26] E. Haghverdi and P. Scott. A categorical model for the geometry of interaction. *Theoretical Computer Science*, 350(2–3):252–274, 2006.

[27] S. Abramsky and B. Coecke. A categorical semantics of quantum protocols. In *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science (LICS '04)*, pp. 415–425. IEEE Computer Society, 2004.